StaffSync

Modern Human Resource Management Solution

Mini Project Report

Technology: MongoDB • Express.js • React.js • Node.js

1. Abstract

StaffSync is a web-based Human Resource Management System (HRMS) designed to make routine people operations easier. It provides a central place to maintain employee information, view workforce summaries, and submit or review leave requests. The application uses MongoDB for data storage, Express and Node.js for REST APIs, and React for the user interface.

2. Problem Statement

Small teams often manage employee records and leave requests using scattered spreadsheets, paper forms, or messages. This can make information difficult to find, update, and track. StaffSync provides a simple centralized interface for these tasks.

3. Objectives

Maintain employee profiles in one place; search employees; submit and track leave requests; approve or reject requests; show dashboard summaries; and demonstrate a full-stack MERN application.

4. Scope

The academic version includes employee CRUD operations, leave-request creation and status updates, search, summary cards, and a responsive dashboard. Authentication, payroll, attendance hardware integration, email notifications, and production access controls are outside the current demo scope.

5. Technology Used

React provides the component-based frontend. Vite runs and builds the frontend. Node.js runs the backend. Express provides REST API routes. MongoDB stores documents. Mongoose defines schemas and database operations. HTML, CSS, and JavaScript support the user interface.

6. System Architecture

The user interacts with the React UI in a browser. The frontend sends HTTP requests to the Express API. The API validates basic request data and uses Mongoose to read/write employee and leave documents in MongoDB. API responses are rendered in the dashboard.

7. Main Modules

Dashboard: workforce and leave summary cards. Employee Management: add, list, search, update through API, and delete employee records. Leave Management: create requests and approve/reject pending requests. REST API: connects the frontend to models. Database: Employee and Leave collections.

8. Database Design

Employee fields: _id, name, email, department, role, status, joiningDate, createdAt, updatedAt. Leave fields: _id, employee reference, leaveType, startDate, endDate, reason, status, createdAt, updatedAt. Relationship: one employee can have many leave requests (one-to-many).

9. Functional Requirements

The system displays summary counts; allows employee creation and searching; allows employee deletion; allows leave requests for existing employees; rejects leave dates where the end date precedes the start date; and allows pending leave requests to be approved or rejected.

10. Non-Functional Requirements

The interface should be usable on desktop and mobile screens. Common API failures should return clear messages. Data should persist in MongoDB. The code should remain modular with separate frontend, routes, and models.

11. User Flow

The admin opens the dashboard. The dashboard fetches employee and leave data. The admin adds an employee, creates a leave request, then reviews it in Leave Management. The admin approves or rejects the request. Dashboard counts refresh after successful operations.

12. Roles and Responsibilities

Workspace Admin maintains employee records and reviews leave requests. Employee profiles are stored and linked to leave requests. The current academic demo does not include separate employee login or authentication. The system validates basic input and calculates summary counts.

13. MVC Pattern

Model: Mongoose Employee and Leave schemas. View: React dashboard, tables, forms, and status labels. Controller/request handling: Express route handlers validate requests, call models, and return JSON responses.

14. Testing Plan

Check the health endpoint; add an employee; verify duplicate email handling; search by name or department; create a leave request; test an invalid date range; approve and reject requests; and check the layout at desktop and mobile widths.

15. Limitations and Future Enhancements

The demo does not include login, role-based permissions, payroll, attendance, email alerts, or automated tests. Future enhancements include JWT authentication, separate admin/employee access, pagination, audit logs, payroll, attendance, and deployment.

16. Conclusion

StaffSync demonstrates how a MERN application can centralize employee records and leave management. The modular frontend, REST API, and MongoDB data models provide a foundation that can be extended with additional HR functions.

17. References

React documentation (react.dev); Express documentation (expressjs.com); MongoDB documentation (mongodb.com/docs); Mongoose documentation (mongoosejs.com/docs); Vite documentation (vite.dev).